

DATA SCIENCE LEARNING NOTES

DAY 03 • LOOPS IN PYTHON

Student: Muhammad Ahmad

Goal: Data Scientist

Journey: 60-Day Intensive Data Science Learning

Day 3 Overview

Loops allow you to repeat operations over sequences and collections. In Data Science, loops help you understand repeated data processing before moving to NumPy, Pandas, and vectorized operations.

Learning Flow

for loop → range() → while loop → break → continue → nested loops → enumerate() → dictionaries → practical data processing

1. What is a Loop?

A loop executes a block of code repeatedly.

```
for i in range(5):  
    print("Hello")
```

2. for Loop

A for loop processes items from a sequence one by one.

```
names = ["Ahmad", "Ali", "Usman"]  
  
for name in names:  
    print(name)
```

3. range()

range() generates numbers. The stop value is not included.

```
for number in range(5):  
    print(number)  
  
for number in range(1, 6):  
    print(number)
```

4. range(start, stop, step)

The step controls the gap between generated values.

```
for number in range(0, 11, 2):  
    print(number)  
  
for number in range(10, 0, -1):  
    print(number)
```

5. Loop with Conditions

Loops and conditions can work together to classify data.

```
marks = [45, 78, 90, 32, 65]  
  
for mark in marks:  
    if mark >= 50:  
        print(mark, "Pass")  
    else:  
        print(mark, "Fail")
```

6. Calculate Total and Average

A loop can accumulate values into a total.

```
marks = [70, 80, 90, 60, 75]
```

```
total = 0

for mark in marks:
    total = total + mark

average = total / len(marks)

print("Total:", total)
print("Average:", average)
```

7. while Loop

A while loop continues while its condition is True. Update the control variable so the loop can stop.

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

8. break

break immediately stops the loop.

```
for number in range(1, 11):
    if number == 6:
        break
    print(number)
```

9. continue

continue skips the current iteration and moves to the next one.

```
for number in range(1, 6):
    if number == 3:
        continue
    print(number)
```

10. Nested Loops

A nested loop is a loop inside another loop.

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

11. enumerate()

enumerate() provides an index and a value.

```
courses = ["Python", "SQL", "Statistics"]

for index, course in enumerate(courses, start=1):
    print(index, course)
```

12. Loop Through a Dictionary

items() provides each dictionary key and value.

```
student = {
    "name": "Ahmad",
    "age": 22,
    "cgpa": 3.23
}

for key, value in student.items():
    print(key, ":", value)
```

13. List of Dictionaries

A list of dictionaries is a useful bridge toward tabular data and Pandas.

```
students = [
    {"name": "Ahmad", "marks": 85},
    {"name": "Ali", "marks": 72},
```

```
{"name": "Usman", "marks": 45}
]

for student in students:
    print(student["name"], student["marks"])
```

14. Find Highest and Lowest

Initialize with the first value, then compare every value.

```
marks = [70, 85, 92, 65, 88]

highest = marks[0]
lowest = marks[0]

for mark in marks:
    if mark > highest:
        highest = mark
    if mark < lowest:
        lowest = mark

print("Highest:", highest)
print("Lowest:", lowest)
```

15. Count Passing and Failing Students

Counters increase when a condition is satisfied.

```
marks = [45, 78, 90, 32, 65, 88]

pass_count = 0
fail_count = 0

for mark in marks:
    if mark >= 50:
        pass_count += 1
    else:
        fail_count += 1

print("Passing:", pass_count)
print("Failing:", fail_count)
```

16. Practical Student Analyzer

This combines Day 2 conditions with Day 3 loops.

```
students = [
    {"name": "Ahmad", "marks": 85},
    {"name": "Ali", "marks": 72},
    {"name": "Usman", "marks": 45},
    {"name": "Hamza", "marks": 91}
]

for student in students:
    if student["marks"] >= 80:
        performance = "Excellent"
    elif student["marks"] >= 70:
        performance = "Good"
    elif student["marks"] >= 50:
        performance = "Pass"
    else:
        performance = "Fail"

    print(student["name"], ":", performance)
```

DAY 3 PRACTICE

Practice 1: Use [10, 20, 30, 40, 50] to calculate total and average with a loop.

Practice 2: For [35, 55, 72, 48, 90, 67, 30], print passing marks, failing marks, and both counts.

Practice 3: For [12, 45, 7, 89, 23, 91, 34], find the highest and lowest values using a loop.

Practice 4: Use a list of student dictionaries to display each student's name, marks, and Pass/Fail status.

DAY 3 MINI PROJECT

Build a Student Performance Analyzer. Calculate total students, total marks, average marks, highest marks, lowest marks, passing students, failing students, and performance for every student.

```
students = [
{"name": "Ahmad", "marks": 85},
{"name": "Ali", "marks": 72},
{"name": "Usman", "marks": 45},
{"name": "Hamza", "marks": 91},
{"name": "Bilal", "marks": 58}
]

total_marks = 0
highest = students[0]["marks"]
lowest = students[0]["marks"]
passed = 0
failed = 0

for student in students:
marks = student["marks"]
total_marks += marks

if marks > highest:
highest = marks

if marks < lowest:
lowest = marks

if marks >= 50:
passed += 1
else:
failed += 1

average = total_marks / len(students)

print("Total Students:", len(students))
print("Total Marks:", total_marks)
print("Average Marks:", average)
print("Highest Marks:", highest)
print("Lowest Marks:", lowest)
print("Passing Students:", passed)
print("Failing Students:", failed)
```

Quick Revision

```
for → repeat over a sequence
range() → generate a number sequence
while → repeat while a condition is True
break → stop the loop
continue → skip the current iteration
nested → loop inside another loop
enumerate → index + value
items() → dictionary key + value
```

Data Science Connection

Loops teach the basic idea of applying an operation repeatedly to data. Later, NumPy and Pandas will let you perform many operations more efficiently through vectorized and tabular operations.

DAY 3 COMPLETE

Day 3 is complete. Next topic: Day 4, Python Functions. You will learn parameters, arguments, return values, default arguments, scope, reusable functions, and practical Data Science functions.